

Laboratorio I

a.a. 2025/2026

Esercitazione su classi, eccezioni, moduli,
e classi di sistema

Esercizio 1: Classe Grade

Vogliamo realizzare un sistema per la gestione dei voti di uno studente universitario. Si scriva la classe **Grade** per rappresentare un esame sostenuto, in termini delle seguenti proprietà private:

- nome (**name**)
- crediti (**credits**)
- data (**date**)
- voto (**grade**)
- lode (**honors**)

La classe deve fornire i metodi

- **equals(g)**, per verificare se un esame sia uguale ad un altro (ovvero abbia lo stesso nome, crediti, data, voto ed eventuale lode)
- **toString()**, per dare una rappresentazione dell'esame sotto forma di stringa

Esercizio 2: Gestione degli errori

Il costruttore della classe **Grade** deve prevedere la seguente gestione degli errori:

- Se viene richiesta la creazione di un esame i cui crediti NON sono un numero positivo, lancia un'eccezione di tipo **InvalidCreditsError**
- Se viene richiesta la creazione di un esame il cui voto NON è un numero compreso tra 18 e 30, lancia un'eccezione di tipo **InvalidValueError**
- Se viene richiesta la creazione di un esame il cui voto ha la lode ma non corrisponde a 30L, lancia un'eccezione di tipo **InvalidValueError**

Esercizio 3: Classe GradeBook

Gli esami (rappresentati come **Grade**) sono raccolti in un **GradeBook**, ovvero in una classe con le proprietà:

- **grades** → array degli esami sostenuti
- **totalCredits** → crediti totali del corso di laurea
- **credits** → crediti conseguiti // dipende da **grades**
- **missingCredits** → crediti da conseguire // dipende dai campi sopra
- **average** → media ponderata dei voti ottenuti // dipende da **grades**
- **startingGrade** → voto di partenza all'appello di laurea

La classe offre inoltre i seguenti metodi:

- **register(grade)** → aggiunta di un voto
- **toString()** → stringa che rappresenta il registro dei voti

Esercizio 4: Bachelor e MasterGradeBook

Si prevedano due classi (analoghe a **GradeBook**) per la registrazione dei voti di

- un corso di laurea triennale (classe **BachelorGradeBook**), per un totale di 180 crediti
- un corso di laurea magistrale (classe **MasterGradeBook**), per un totale di 120 crediti

Esercizio 5: Gestione degli errori nei GradeBook

Le classi **GradeBook**, **BachelorGradeBook** e **MasterGradeBook** devono prevedere la seguente gestione degli errori:

- Se l'input passato a **register** non è un esame (**Grade**), deve lanciare un'eccezione di tipo **TypeError** (tipo predefinito)
- Se viene richiesta la registrazione di un esame già presente, viene lanciata un'eccezione di tipo **DuplicatedGradeError**
- Se viene richiesto il voto di laurea di partenza (**startingGrade**) e i crediti totali non sono stati raggiunti, deve lanciare un'eccezione di tipo **MissingCreditsError**

Esercizio 6: Lettura e scrittura su file

Modificare la classe `GradeBook` fornendo i metodi:

- `exportJSON(f)`, salva i dati dell'oggetto `GradeBook` in un file JSON al percorso specificato da `f`
- `fromJSON(f)`, metodo statico che carica i dati da un file JSON situato al percorso `f` e restituisce un nuovo oggetto `GradeBook` con i contenuti caricati

(per casa) **Esercizio 7: Definire un modulo**

Creare un modulo `gradebook.js` che esporti le dichiarazioni di `Grade`, `GradeBook`, `BachelorGradeBook`, `MasterGradeBook`, e tutte le classi di eccezioni appena implementate.

(per casa) **Esercizio 8:** Validazione codice di matricola

Scrivere una funzione `matricolaValida(m)` che utilizzi espressioni regolari per verificare se una matricola `m` è corretta, ossia è composta da esattamente 6 cifre.

Esempi:

- `matricolaValida("01234") → false`
- `matricolaValida("A01234") → false`
- `matricolaValida("012345") → true`

(per casa) **Esercizio 9: Validazione email studenti**

Scrivere una funzione `emailStudenteUnipiValida(e)` che utilizzi espressioni regolari per verificare se e segue il formato: `n.cognome@studenti.unipi.it`, ossia:

- L'iniziale del nome deve essere una singola lettera minuscola
- Il cognome deve essere composto da sole lettere minuscole
- Il suffisso deve essere esattamente `@studenti.unipi.it`
- Nota: alla fine del cognome può essere aggiunto un numero ≥ 1 per i casi di omonimia

Esempi:

- `emailStudenteUnipiValida("g.rossi@studenti.unipi.it")` → `true`
- `emailStudenteUnipiValida("b.bianchi1@studenti.unipi.it")` → `true`
- `emailStudenteUnipiValida("j.marino13@studenti.unipi.it")` → `true`
- `emailStudenteUnipiValida("test")` → `false`
- `emailStudenteUnipiValida("k.costa@unipi.it")` → `false`
- `emailStudenteUnipiValida("c.ferrari0@studenti.unipi.it")` → `false`
- `emailStudenteUnipiValida("d.esposito@studenti.unifi.it")` → `false`